

8th Week Internet Programming Working Questions

JavaScript Events, DOM, Data Attributes, and Browser Extensions

Name: _____

Student No: _____

Instructions

- Answer all multiple-choice questions by selecting the best option.
- Answer the coding questions clearly and write clean, working code.
- Total: **100 points**.
- Multiple-choice questions: **15 questions** $\times$ **4 points** = **60 points**
- Coding Question 1: **15 points**
- Coding Question 2: **25 points**

Part A — Multiple-Choice Questions (60 points)

- Q1.** If a click event listener is attached to a `div`, and the user clicks an `img` inside that `div`, what happens?
- A) Only the `img` receives the event.
 - B) The event listener on the `div` can still fire.
 - C) The browser throws an error.
 - D) Events work only on buttons.
- Q2.** In JavaScript events, `event.target` refers to:
- A) The element on which the listener was attached
 - B) The root HTML element
 - C) The element that was actually clicked
 - D) The first child of the clicked element
- Q3.** In JavaScript events, `event.currentTarget` refers to:
- A) The element that originally dispatched the event
 - B) The element that the event handler is currently attached to
 - C) The last element in the DOM tree
 - D) The selected CSS class

- Q4.** If both an inner `div` and an outer `div` have click listeners, and the user clicks the inner `div`, which one fires first by default?
- A) The outer `div`
 - B) The inner `div`
 - C) Both fire at exactly the same time
 - D) Neither fires
- Q5.** What is the name of the default process in which an event moves from the inner element toward outer ancestors?
- A) Event hiding
 - B) Event bubbling
 - C) Event cloning
 - D) Event styling
- Q6.** Which JavaScript method is used to stop an event from continuing to bubble upward?
- A) `event.preventDefault()`
 - B) `event.stopPropagation()`
 - C) `event.removeListener()`
 - D) `event.endBubble()`
- Q7.** Which option enables capture-phase listening in `addEventListener`?
- A) `{ target: true }`
 - B) `{ bubble: false }`
 - C) `{ capture: true }`
 - D) `{ inner: true }`
- Q8.** What does the following line do?
- ```
const boxes = document.querySelectorAll('#grid div');
```
- A) Selects only one `div` inside `#grid`
  - B) Removes all `div` elements
  - C) Selects all `div` elements inside `#grid`
  - D) Creates new `div` elements inside `#grid`
- Q9.** What is the purpose of `appendChild()` in DOM programming?
- A) It deletes a child element
  - B) It attaches a node as the last child of a parent
  - C) It renames an HTML tag
  - D) It changes CSS automatically

- Q10.** What is the result range of the following expression if `freeBoxes.length` is 5?
- ```
Math.floor(Math.random() * freeBoxes.length)
```
- A) 1 to 5
 - B) 0 to 5
 - C) 0 to 4
 - D) 1 to 4
- Q11.** What is the main purpose of HTML `data-*` attributes?
- A) To add custom data to HTML elements
 - B) To replace JavaScript functions
 - C) To create new HTML tags
 - D) To automatically style elements
- Q12.** If an element has `data-index="7"`, how can JavaScript access this value using `dataset`?
- A) `element.dataset[7]`
 - B) `element.dataset.data-index`
 - C) `element.dataset.index`
 - D) `element.index.dataset`
- Q13.** Why can `childNodes` sometimes return more items than expected?
- A) Because it includes text nodes such as whitespace
 - B) Because it includes only CSS rules
 - C) Because it removes HTML comments
 - D) Because it duplicates each child element
- Q14.** In the DOM, the `parentNode` of a `section` element usually refers to:
- A) Its first child
 - B) The element that contains that `section`
 - C) The document title
 - D) The browser window
- Q15.** In the lecture examples, jQuery `.on()` and `.off()` are most similar to which standard DOM methods?
- A) `createElement()` and `appendChild()`
 - B) `addEventListener()` and `removeEventListener()`
 - C) `querySelector()` and `querySelectorAll()`
 - D) `textContent` and `innerHTML`

Part B — Coding Questions (40 points)

Q16. Event Bubbling and `stopPropagation()` (15 points)

Create a small example with nested `div` elements. When the user clicks the inner box, the page should first show the default bubbling behavior, and then you should modify the code so that only the inner click handler runs.

Tasks:

1. Create an outer `div` and an inner `div`.
2. Style them clearly using CSS so both are visible.
3. Attach click listeners to both elements.
4. Print different messages to the console for the outer and inner clicks.
5. Modify the JavaScript so that clicking the inner element no longer triggers the outer click handler.

Write your HTML, CSS, and JavaScript below:

Starter structure:

```
1 <!-- index.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Event Bubbling Demo</title>
8   <link rel="stylesheet" href="style.css">
9 </head>
10 <body>
11   <div id="outer">
12     Click me!
13     <div id="inner">No, click me!</div>
14   </div>
15
16   <script src="script.js"></script>
17 </body>
18 </html>
```

```
1  /* style.css */
2  #outer {
3      width: 320px;
4      padding: 30px;
5      background-color: #dbeafe;
6      border: 2px solid #1d4ed8;
7  }
8
9  #inner {
10     margin-top: 20px;
11     padding: 20px;
12     background-color: #fde68a;
13     border: 2px solid #b45309;
14 }
```

```
1  // script.js
2  const outer = document.querySelector('#outer');
3  const inner = document.querySelector('#inner');
4
5  function onOuterClick(event) {
6      console.log('Outer clicked');
7  }
8
9  function onInnerClick(event) {
10     console.log('Inner clicked');
11     // Add one line here so the event does not bubble upward
12 }
13
14 outer.addEventListener('click', onOuterClick);
15 inner.addEventListener('click', onInnerClick);
```

Expected outcome: The student should demonstrate both default bubbling and the corrected version using `event.stopPropagation()`.

Q17. Modify the JavaScript File for Tic-Tac-Toe (25 points)

You are given the following HTML and CSS files. **Do not change the HTML or CSS structure.** Modify only the `script.js` file so that:

1. When a user clicks an empty box, an X appears in that box.

2. The same box cannot be clicked again.
3. After the user move, the computer randomly selects one empty box and places an O.
4. The program uses the `data-index` attribute to track the box number.
5. The program stores box ownership in a JavaScript object named `takenBoxes`.
6. At the end of each turn, if there is a winner, the page should display the result inside the `#results` area.

Given HTML file:

```
1 <!-- index.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Tic-Tac-Toe</title>
8   <link rel="stylesheet" href="style.css">
9 </head>
10 <body>
11   <h1>Tic-Tac-Toe</h1>
12
13   <div id="grid">
14     <div data-index="0"></div>
15     <div data-index="1"></div>
16     <div data-index="2"></div>
17     <div data-index="3"></div>
18     <div data-index="4"></div>
19     <div data-index="5"></div>
20     <div data-index="6"></div>
21     <div data-index="7"></div>
22     <div data-index="8"></div>
23   </div>
24
25   <div id="results"></div>
26
27   <script src="script.js"></script>
28 </body>
29 </html>
```

Given CSS file:

```
1  /* style.css */
2  body {
3      font-family: Arial, sans-serif;
4      text-align: center;
5  }
6
7  #grid {
8      width: 306px;
9      margin: 20px auto;
10     display: grid;
11     grid-template-columns: repeat(3, 100px);
12     grid-template-rows: repeat(3, 100px);
13     gap: 3px;
14 }
15
16 #grid div {
17     border: 2px solid #444;
18     display: flex;
19     justify-content: center;
20     align-items: center;
21     font-size: 42px;
22     font-weight: bold;
23     cursor: pointer;
24     background-color: #f8fafc;
25 }
26
27 #results {
28     margin-top: 20px;
29     font-size: 24px;
30     font-weight: bold;
31 }
```

Modify this JavaScript file:

```
1  // script.js
2  const boxes = document.querySelectorAll('#grid div');
3  const freeBoxes = [];
4  const takenBoxes = {};
5
6  for (const box of boxes) {
7      freeBoxes.push(box);
8      box.addEventListener('click', changeToX);
9  }
```

```
10
11 function changeToX(event) {
12     // Student will complete this function
13 }
14
15 function computerChooseO() {
16     // Student will complete this function
17 }
18
19 function assignSpace(space, owner) {
20     // Student will complete this function
21 }
22
23 function getWinner() {
24     // Student will complete this function
25 }
26
27 function checkBoxes(a, b, c) {
28     // Student will complete this function
29 }
30
31 function displayWinner() {
32     // Student will complete this function
33 }
```

Requirements for modification:

1. Use `event.currentTarget` inside `changeToX()`.
2. Use `dataset.index` and convert it to a number if needed.
3. Add the player symbol to the clicked box.
4. Remove the click listener after the box is assigned.
5. Remove the assigned box from `freeBoxes`.
6. Make the computer choose a random free box.
7. Update `takenBoxes` after each move.
8. Check rows, columns, and diagonals for a winner.
9. Show `You win!`, `Computer wins!`, or `Tie!` in `#results`.

Note: This question measures the student's ability to modify an existing JavaScript file by using event handling, DOM selection, arrays, objects, and data attributes together.

End of Week 8 Working Questions